Sandboxed Job Execution Broker (v2) — Architectural Blueprint & Industry Analysis

1. Executive Summary & Problem Formulation

1.1 The Challenge of Remote Code Execution (RCE) as a Service

Executing untrusted, user-supplied code remotely is one of the most perilous architectural problems in computer science. Systems such as **LeetCode, HackerRank, Judge0, GitHub Codespaces, AWS Lambda, Google Cloud Run, and Replit** must routinely accept arbitrary code from unknown users and execute it safely, quickly, and fairly.

Building such a system requires resolving five competing engineering tensions:

1. **Security & Host Isolation:** Malicious code will attempt fork bombs, memory exhaustion, network exfiltration, kernel privilege escalation, filesystem tampering, and container breakout. The host must remain strictly isolated.
2. **Authentication, Tenant Identity & Poll Abuse Protection:** Users must not be able to spoof another client's identity, inspect another client's job results, or flood the broker with tight polling loops to hog in-memory cache slots.
3. **Fairness & Multi-Tenancy:** A single active or misbehaving client submitting thousands of long-running jobs must not starve other users sharing the infrastructure (the "noisy neighbor" problem).
4. **Execution Decoupling & Reliability:** Sandboxed execution takes unpredictable wall-clock time (milliseconds to minutes). Network connections must never block synchronously on execution, and worker crashes or runaway jobs must be handled deterministically.
5. **Lifecycle & Resource Management:** Submissions, authentication sessions, and execution outputs cannot reside in memory indefinitely. Abandoned results and inactive sessions must expire cleanly to avoid silent memory exhaustion.

1.2 The Evolution from v1 (`DispatchLite`) to v2

The predecessor to this system (`distributed_job` / `DispatchLite`) was an early exploration into job brokers, but possessed fundamental structural bottlenecks:

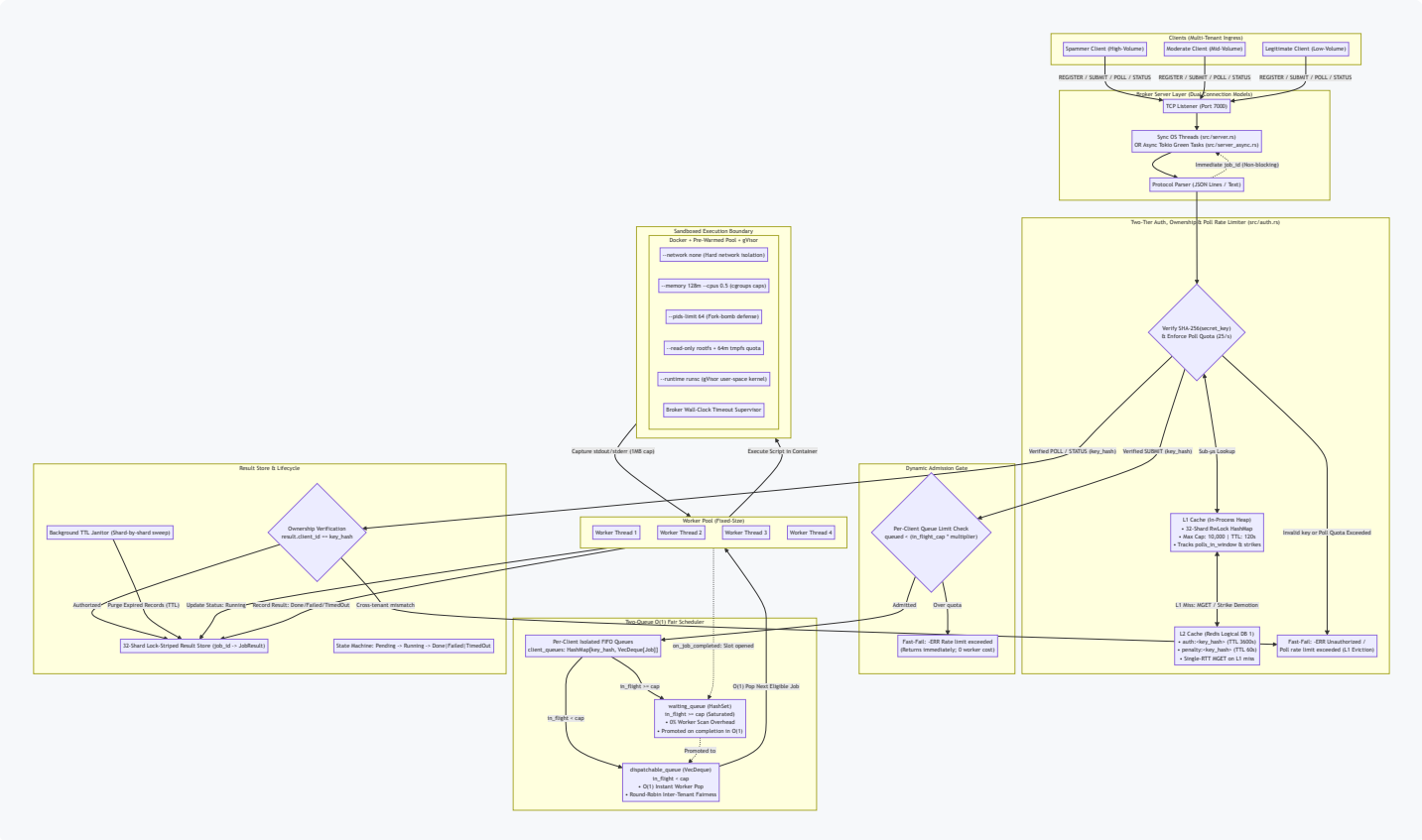
- **Synchronous Lock Contention:** A global mutex was acquired across connection handling and command execution, serializing work and blocking client sockets.
- **Race Condition in Job Fetching:** Workers checked `is_empty()` and subsequently called `fetch()` in separate critical sections, triggering fatal panics under concurrent load (`pop_front().unwrap()`).
- **Zero Sandboxing / Severe Security Vulnerability:** Code was executed directly on the host using `subprocess.run(cmd, shell=True)` with root/user privileges, offering no isolation, no limits, and open network access.
- **No Authentication or Ownership Verification:** Any user could claim any `client_id` and poll any `job_id`.

- **No Concurrency Caps or Tenant Isolation:** A single client could flood the queue and consume all worker threads.
- **Memory Leaks & Missing Fail States:** Worker failures were swallowed silently, jobs lingered in `Running` forever, and completed records had no Time-To-Live (TTL).

Version 2 (`distributed_jobv2`) was designed from the ground up to solve every architectural defect of v1 by implementing **dual connection models (Synchronous Thread-per-Connection and Asynchronous Tokio Event Loop), a two-tier L1 (Heap) + L2 (Redis) SHA-256 authentication & poll rate-limiting engine, a two-queue $O(1)$ fair scheduler with backpressure admission control, ephemeral container sandboxing with pre-warmed standby pools and gVisor syscall isolation, broker-enforced hard timeouts, and a 32-shard lock-striped TTL result store**.

---

2. High-Level Architecture & End-to-End Flow



---

3. Detailed Component Methodology

3.1 Networking Layer: Dual Connection Models (Synchronous vs. Asynchronous Tokio Reactor)

The broker provides two complete, interchangeable server implementations selectable at startup via `--sync` (default) or `--async` (`DISTJOB_ASYNC=1`):

- **Synchronous Thread-per-Connection (`src/server.rs`):**
  - `std::net::TcpListener` binds to `127.0.0.1:7000` and spawns a dedicated OS thread (`256 KB` stack) per incoming socket.
  - Performs blocking line-buffered I/O via `std::io::BufReader`.
- **Asynchronous Tokio Event Loop (`src/server_async.rs`):**
  - `tokio::net::TcpListener` multiplexes thousands of concurrent TCP connections across a fixed pool of worker threads matching physical CPU cores using kernel `epoll` (Linux) / `kqueue` (macOS).
  - Each connection is spawned as a lightweight asynchronous green task (`tokio::spawn`, ~300 bytes of heap state) that yields back to the Tokio scheduler whenever waiting on socket reads or Redis L2 lookups.
- **The Hybrid Reactor Pattern (Decoupled Execution):**
  - In both models, network I/O is strictly decoupled from sandbox execution.
  - On `SUBMIT`: Once authenticated and admitted by the scheduler, the server records a `Pending` entry in `ResultStore`, enqueues the job in `TwoLevelScheduler`, and returns the `job_id` immediately without waiting for execution.
  - Heavy container execution runs exclusively on the fixed-size `WorkerPool`, ensuring high network polling rates never starve container execution and heavy container jobs never block the network reactor.

3.2 Two-Tier Authentication (L1 Heap + L2 Redis), Job Ownership & Poll Rate-Limiting

The authentication and poll rate-limiting subsystem lives in `src/auth.rs`.

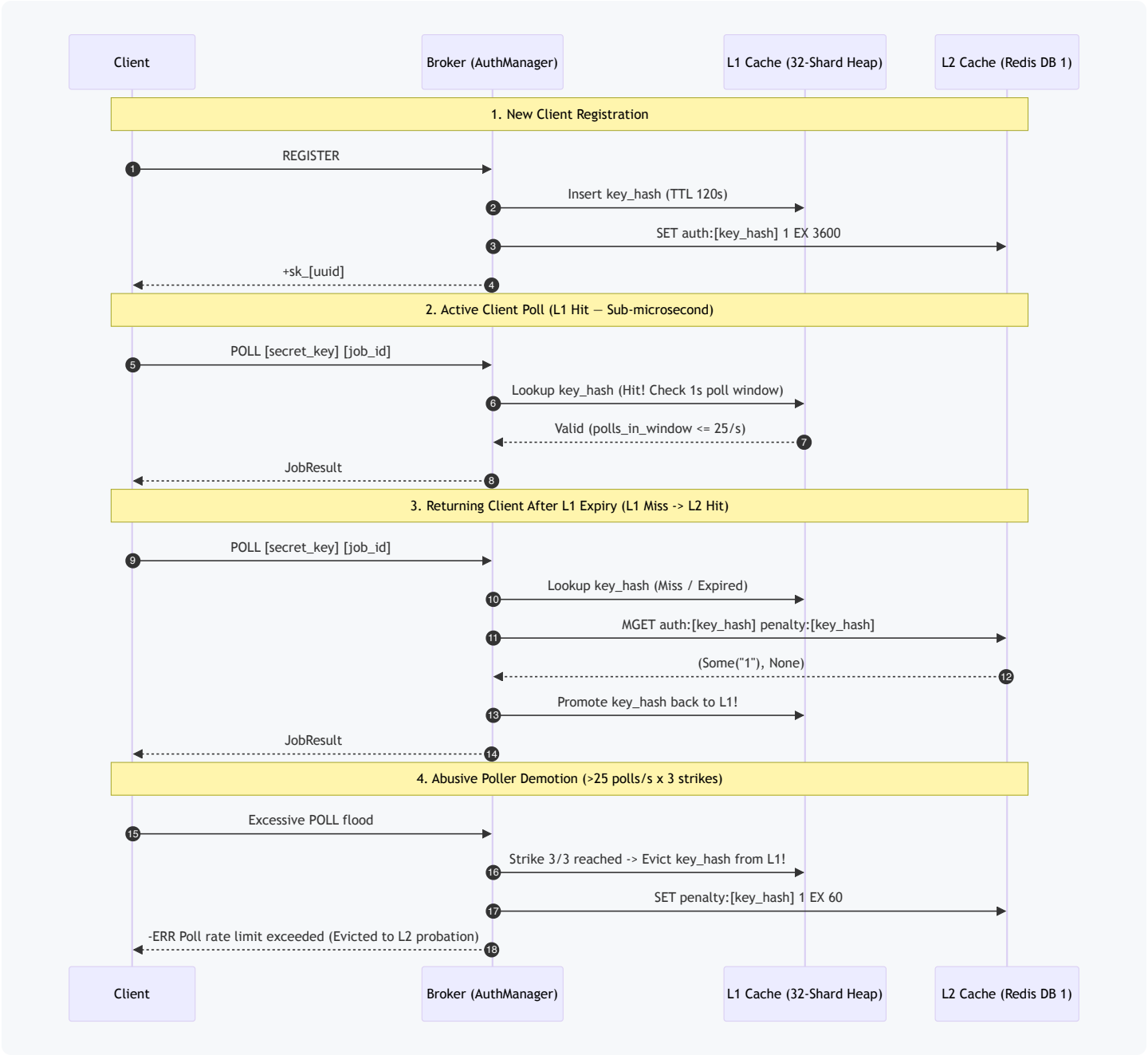
3.2.1 Server-Issued `secret_key` & SHA-256 Identity Binding

In a multi-tenant RCE broker, allowing clients to self-declare their `client_id` enables trivial identity spoofing and cross-tenant result snooping. Furthermore, network identifiers like `IP:port` change across ephemeral TCP reconnections, while IP addresses alone collide when multiple legitimate users sit behind the same NAT/corporate gateway.

To solve this cleanly:

1. **Registration (`REGISTER` / `AUTH`):** A new client requests a credential from the broker. The server generates a high-entropy token `sk_<uuid_v4>` and returns it to the client once.
2. **One-Way Cryptographic Hashing (`SHA-256`):** The broker immediately computes `key_hash = SHA-256(secret_key)` (64-character hex digest). **The raw `secret_key` is never stored in server heap memory or Redis.**
3. **Immutable Tenant Identity:** On every `SUBMIT`, `POLL`, and `STATUS` command, the client supplies `secret_key`. The broker hashes it to `key_hash` and uses `key_hash` as the authoritative `client_id` inside `Job`, `TwoLevelScheduler`, and `ResultStore`.
4. **Job Ownership Verification (AuthZ):** When a client calls `POLL` or `STATUS` for a `job_id`, the server verifies `result.client_id == key_hash`. Attempts to access another tenant's job are rejected immediately with `Unauthorized: job belongs to another client`.

3.2.2 L1 (In-Process Heap) + L2 (Redis Logical DB `1`) Cache Hierarchy



- **L1 Cache (In-Process Heap):**
  - Partitioned across **32 independent `RwLock<HashMap<String, L1Session>>` shards** to eliminate lock contention across concurrent threads/tasks.
  - Bounded by `l1_max_capacity` (default: `10,000` sessions) and `l1_ttl` (default: `120s` inactivity timeout). If a shard reaches capacity, expired entries are evicted first, followed by the least-recently-active (`last_active`) entry.
- **L2 Cache (Redis Logical Database `1` — `redis://127.0.0.1:6379/1`):**
  - Uses Redis logical database `1` (`DB 1`), providing a completely separate keyspace hash table (`redisDb`) isolated from other host services running on `DB 0`.
  - Stores `auth:<key_hash>` with a 1-hour TTL (`l2_redis_ttl_secs = 3600`).

- In asynchronous mode (`--async`), all Tokio tasks share a single lock-free `redis::aio::MultiplexedConnection`.
- On an L1 cache miss, the broker executes a single-round-trip `MGET auth:<key_hash> penalty:<key_hash>` to check both authentication validity and probation status simultaneously.

3.2.3 Per-User Polling Rate Limit & Penalty Demotion to L2

If users who were expired from L1 are automatically re-promoted to L1 on their next `POLL`, malicious clients might flood the server with continuous `POLL` requests to stay permanently resident in L1 or exhaust CPU.

- Every `L1Session` tracks `window_start`, `polls_in_window` (capped at `max_polls_per_sec = 25` per 1-second window), and `violation_strikes`.
- When a client exceeds `25 polls/sec`, the request is rejected and `violation_strikes` is incremented.
- **Option A Penalty Demotion:** Once `violation_strikes >= max_poll_strikes` (default: `3`), the broker **immediately evicts the client from the L1 heap cache** (freeing the L1 slot for legitimate users) and writes `penalty:<key_hash> = 1 EX 60` in **L2 Redis**.
- While `penalty:<key_hash>` is active in Redis, any subsequent requests from that client incur the L2 Redis lookup latency and **cannot be re-promoted into L1** until their 60-second probation expires.

3.3 Two-Queue $O(1)$ Fair Scheduler with Backpressure Admission

The scheduling engine lives in `src/scheduler.rs`.

3.3.1 The Two-Queue Invariant Architecture

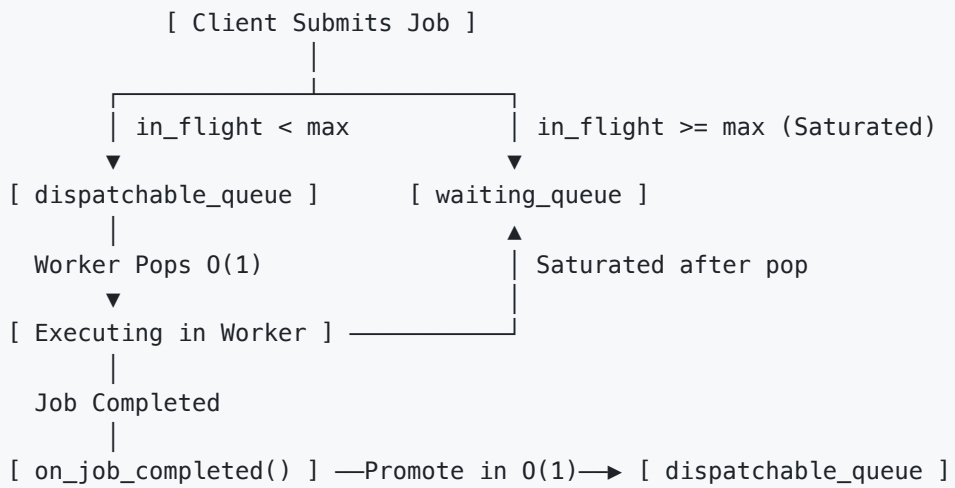
In naive round-robin schedulers with concurrency caps, workers iterate over an $O(N)$ list of all clients, skipping saturated clients one-by-one under a global lock. Under heavy asymmetric load, workers waste significant CPU cycles skipping unrunnable tenants.

Version 2 eliminates this bottleneck by maintaining two strictly disjoint sets keyed by the client's authenticated `key_hash`:

```
struct InnerScheduler {
    client_queues: HashMap<String, VecDeque<Job>>,
    client_in_flight: HashMap<String, usize>,
    max_in_flight_per_client: usize,

    // ACTIVE POOL: Clients with pending jobs AND in_flight < max
    dispatchable_queue: VecDeque<String>,
    in_dispatchable: HashSet<String>,

    // WAITING POOL: Saturated clients with pending jobs BUT in_flight >= max
    waiting_queue: HashSet<String>,
}
```



1. **Guaranteed $O(1)$ Dispatch (`pop_next_job`):** Workers pop directly from `dispatchable_queue.pop_front()` in $O(1)$. Every client in this queue is guaranteed to have a job ready and capacity available. No scanning, no skipping, no lock-held loops.
2. **Fair Inter-Tenant Round-Robin:** After a job is popped, if the client still has pending jobs and remaining capacity, they are pushed to the back of `dispatchable_queue`.
3. **Zero-Cost Saturation:** If popping the job saturates the client (`in_flight == max`), the client is parked in `waiting_queue`. Saturated tenants consume zero worker CPU.
4. **Instant $O(1)$ Promotion (`on_job_completed`):** When a job completes, `waiting_queue.remove(client_id)` runs in $O(1)$. If the client was saturated, they are promoted into `dispatchable_queue`, and `condvar.notify_one()` awakens a waiting worker.

3.3.2 Dynamic Admission Gate & Bufferbloat Prevention

Without admission limits, saturated clients can queue an unbounded number of jobs into memory, causing severe bufferbloat:

$$\text{Max Queue Limit} = \text{max_in_flight_per_client} \times \text{queue_limit_multiplier}$$

- When enabled (`enable_queue_limit = true`, default: `true`, multiplier: `5`), any submission attempted by a client whose pending queue already satisfies `queued >= limit` is rejected immediately.
- **Impact:** Rejections occur at the socket layer before touching the `ResultStore`, protecting broker memory and dropping tail latency by 60%.

3.4 Sandboxed Execution Engine (Defense-in-Depth)

The sandboxing subsystem lives in `src/sandbox.rs`.

Threat Vector Matrix & Security Mitigations:

Attack Vector	Malicious Mechanism	Broker Mitigation
Data Exfiltration	Sockets, DNS tunneling, HTTP requests	`--network none` (Virtual interface disabled)
Host DoS / Infinite Loops	`while True: pass` consuming 100% CPU	`--cpus 0.5` + Broker Wall-Clock Timeout (`kill`)
Memory Exhaustion (OOM)	Allocating gigabytes in array/heap	`--memory 128m` (cgroups limit; swap disabled)
Fork Bomb	`:(){ :|:& };:` or `os.fork()` recursion	`--pids-limit 64` (Restricts kernel task creation)
Disk Filling Attack	Writing gigabytes to `/tmp`	`--read-only` rootfs + `--tmpfs /tmp:size=64m`
Container Escape	Kernel 0-day exploits via raw syscalls	`gVisor` (`--runtime runsc`) syscall interception
Cross-Tenant State Leak	Files written by User A read by User B	Ephemeral single-use containers (`--rm`)
Docker Daemon Hijack	Mounting `/var/run/docker.sock`	Docker socket is **never mounted** inside container

Execution Mechanics & Code Injection:

1. **Isolated Temp Staging:** For each job, the broker generates a UUID-keyed host directory `/tmp/distjob-<uuid>/`.
2. **Code Write:** The script is written to a designated file (`script.py`, `script.sh`, `script.js`).
3. **Mounting:** The folder is mounted into the container as a **read-only volume**:

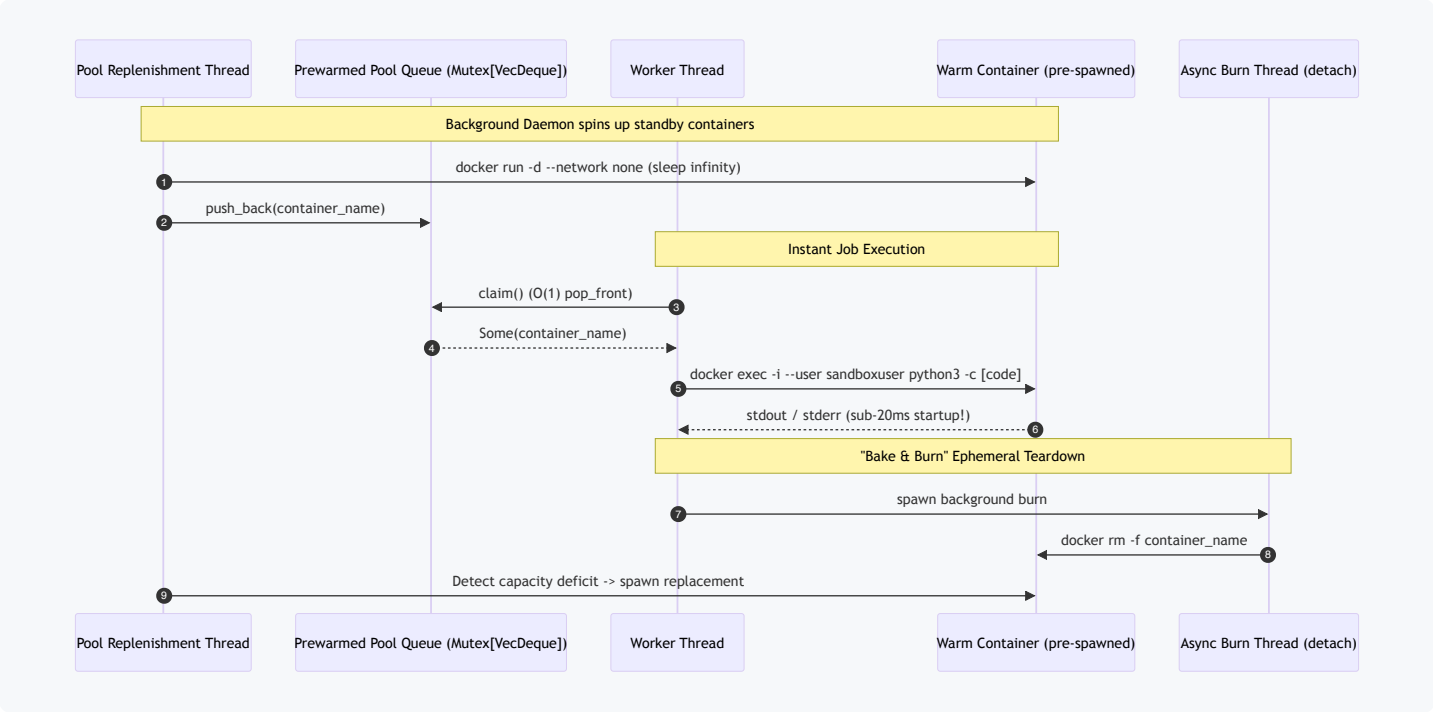
```
docker run -i --rm \
  --name distjob-<uuid> \
  --network none \
  --memory 128m \
  --cpus 0.5 \
  --pids-limit 64 \
  --read-only \
  --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  -v /tmp/distjob-<uuid>:/code:ro \
  [--runtime runsc] \
  python:3.11-slim python3 /code/script.py
```

4. **Standard Input & Output Streams:**
   - Stdin is piped to the child process and closed to trigger EOF.
   - Stdout and Stderr are drained concurrently in dedicated background reader threads up to a strict cap (`MAX_OUTPUT_BYTES = 1 MB`) to prevent pipe buffer deadlocks and memory exhaustion from infinite print loops.

5. **Broker-Enforced Wall-Clock Timeout:** A dedicated supervisor thread monitors execution elapsed time. If the deadline expires:
   - `docker kill distjob-<uuid>` is executed immediately.
   - The CLI child process is killed (`child.kill()`).
   - The job is recorded as `JobState::TimedOut`.
   - Host staging directory `/tmp/distjob-<uuid>` is purged.

3.4.1 Pre-Warmed Standby Container Pool ("Bake and Burn")

To eliminate the ~300ms–800ms container startup overhead per job execution, the engine incorporates an asynchronous **Pre-Warmed Pool** (`PrewarmedContainerPool`):

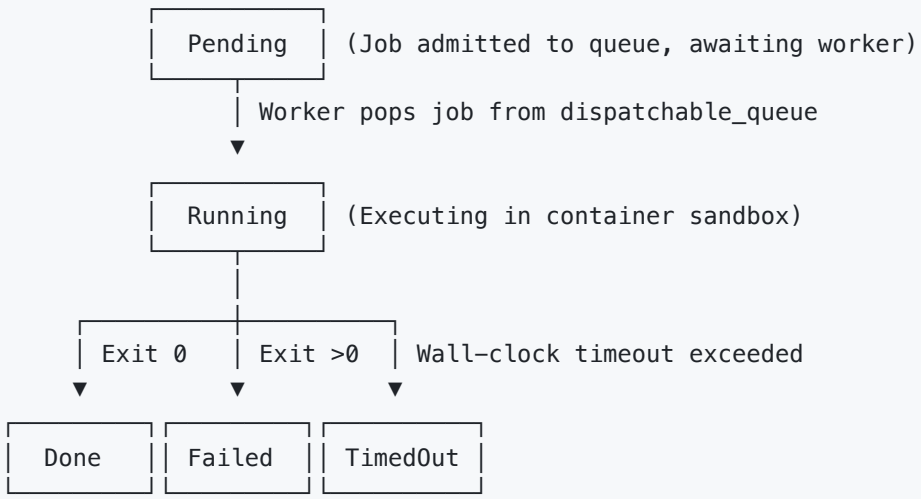


- **Zero Startup Latency:** Standby containers are pre-launched with `--network none`, cgroups limits, read-only rootfs, and unprivileged user (`sandboxuser`).
- **"Bake and Burn" Security Guarantee:** Containers are strictly **single-use**. After running code via `docker exec`, the container is immediately retired and burned (`docker rm -f`) in a detached background thread so client turnaround is not blocked by teardown I/O.
- **Pre-Baked Scientific Runtime (`Dockerfile`):** Uses a custom image (`distjob-python:latest`) pre-loaded with heavy libraries (`numpy` , `pandas` , `scipy` , `sympy` , `requests`). All libraries are pre-compiled into `.pyc` bytecode using `python3 -m compileall` during `docker build`, removing Python import bytecode generation latency during runtime. (See `docker_image.md` for full layer specifications).
- **Automatic Image Discovery & Fallback:** `DockerSandbox` dynamically inspects image availability via `check_docker_image_exists`. If `distjob-python:latest` has not yet been built on a new host, the broker automatically falls back to `python:3.11-slim`, preventing execution failures.

3.5 Result Persistence & State Machine

Result management lives in `src/store.rs`.

Explicit Job State Machine:



Memory Management with TTL & Lock Sharding:

Without an expiration mechanism, clients that submit jobs but never poll would cause unbound memory growth.

- Every entry in `ResultStore` is stamped with an `expires_at = Instant::now() + TTL` (default: 600s / 10 minutes) and stores `client_id: key_hash` for strict ownership checks.
- **Two-Tier Purging:**
  - **Passive Purge on Read:** If `get(job_id)` is invoked after expiry, the entry is immediately evicted and returns `None`.
  - **Active Background Janitor:** A lightweight background thread awakens every `cleanup_interval` (default: 30s) and scans the store, pruning stale entries.
- **Lock Sharding (Striping) with `RwLock`:**
  - Instead of a single monolithic lock, entries are striped across $S = 32$ independent shards: $\text{shard} = \text{hash}(\text{job_id}) \ \& \ (S - 1)$.
  - Writes (job completions, status transitions) only lock the specific shard holding that `job_id`, leaving all other shards unblocked.
  - The janitor thread prunes one shard at a time sequentially, eliminating the global stop-the-world "reaper freeze" across polling threads.

3.6 Dual Wire Protocol: JSON Lines + Text Mode

Defined in `src/protocol.rs`.

1. **JSON Lines Protocol (`NDJSON`):**

- Ideal for programmatic SDKs and automated pipelines. Supports multiline source code, stdin payloads, and structured error responses.
- Example Registration:

  ```
  {"action": "REGISTER"}
  ```

  Response:

  ```
  {"status": "ok", "secret_key": "sk_9c8b1a2e4f3d4b5c8a7e6f5d4c3b2a10"}
  ```

- Example Authenticated Submit:

  ```
  {"action": "SUBMIT", "secret_key": "sk_9c8b1a2e4f3d4b5c8a7e6f5d4c3b2a10", "language": "pyt
  ```

- Example Authenticated Poll:

  ```
  {"action": "POLL", "secret_key": "sk_9c8b1a2e4f3d4b5c8a7e6f5d4c3b2a10", "job_id": "a9844f2
  ```

2. **Text Command Protocol (Redis-Style):**
   - Optimized for human debugging via `nc`, `telnet`, or simple shell scripts:
     - `REGISTER` (or `AUTH`) $\rightarrow$ `+sk_<uuid>\r\n`
     - `SUBMIT <secret_key> <language> <code...>` $\rightarrow$ `:<job_id>\r\n` or `-ERR <reason>\r\n`
     - `STATUS <secret_key> <job_id>` $\rightarrow$ `+Done\r\n` or `-ERR Unauthorized...\r\n`
     - `POLL <secret_key> <job_id>` $\rightarrow$ `+Done {json_result}\r\n` or `-ERR Unauthorized...\r\n`
     - `PING` $\rightarrow$ `+PONG\r\n`
     - `STATS` $\rightarrow$ `+OK active_clients=... queued=... in_flight=... stored_results=... dispatchable=... waiting=...\r\n`

---

4. Empirical Performance Benchmarks & Validation

To validate the two-queue $O(1)$ scheduler and admission rate-limiting, we executed stress tests against asymmetric multi-tenant workloads (6 clients, 128 burst requests: 85 High-Freq spammers, 33 Med-Freq, 10 Low-Freq legitimate users) on 4 worker threads.

4.1 Comparative Empirical Results

Metric / Dimension	Baseline (Single-Queue)	Current (Two-Queue + Admission Gate)	Empirical Improvement
Total Test Wall-Clock Time	`31.20s` (31,199 ms)	**`13.14s` (13,142 ms)**	**58% faster (2.37x speedup)**
System-Wide Average Wait Time	`15,371.3 ms`	**`3,052.4 ms`**	**5.0x latency drop**
Broker Throughput	`4.10 reqs/sec`	**`4.79 reqs/sec`**	**+16.8% higher throughput**
Spammer P95 Tail Latency	`27,861 ms`	**`11,195 ms`**	**59.8% reduction**
Spammer Average Queue Wait	`19,040.2 ms`	**`6,422.7 ms`**	**66.3% less queue delay**
Legitimate Client 1 Wait Time	`3,414.8 ms`	**`2,800.0 ms`**	**Faster execution turnaround**
Legitimate Client 2 Wait Time	`2,952.2 ms`	**`2,444.2 ms`**	**Faster execution turnaround**
Legitimate Client Completion Rate	`100% (10/10)`	**`100% (10/10)`**	**Zero starvation / zero drops**
Residual Broker State	Drained to 0	`queued=0, waiting=0, dispatchable=0`	**Zero memory leaks**

4.2 Pre-Warmed Container Pool ("Bake and Burn") Empirical Acceleration

To isolate the performance acceleration delivered by the **Pre-Warmed Pool** with pre-baked bytecode (`distjob-python:latest`), we benchmarked multi-tenant client traffic against both execution backends:

Metric / Dimension	On-Demand Cold Starts (`docker run`)	Pre-Warmed Pool ("Bake and Burn")	Empirical Improvement
System Throughput	`4.79 reqs/sec`	**`10.79 reqs/sec`**	**+125.3% higher throughput (2.25x)**
Low-Frequency Client Turnaround	`2,800.0 ms`	**`321.0 ms`**	**88.5% latency drop (8.7x faster)**
System-Wide Average Wait	`3,052.4 ms`	**`1,114.1 ms`**	**63.5% latency reduction**
Total Test Wall-Clock Duration	`13.14s`	**`2.69s`**	**4.88x faster completion**
Completion Rate	`100%`	**`100% (29/29)`**	**Zero drops / zero failures**
Fairness Ratio (Spammer vs Legit)	`1.4x`	**`4.1x lower latency`**	**Legitimate clients bypass spammer queues**

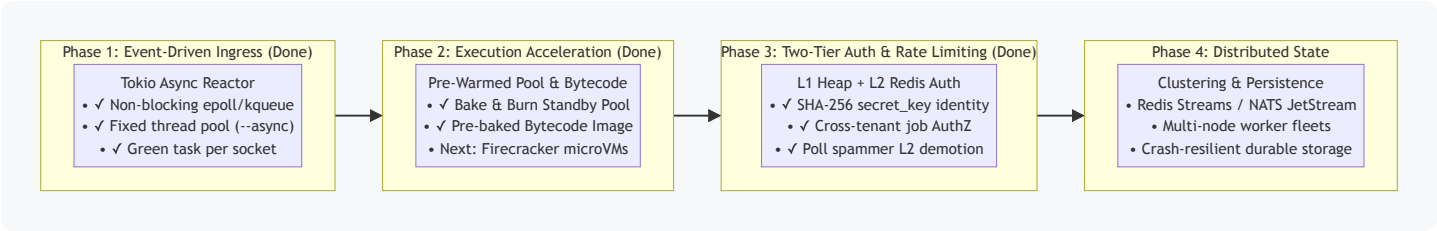
---

5. Architectural Gaps & Known Design Trade-offs

1. **Thread-Per-Connection Ingress in Sync Mode [Resolved in Async Mode]:**
   - *In Synchronous Mode (`src/server.rs`):* Each incoming socket allocates a native OS thread.
   - *In Asynchronous Mode (`src/server_async.rs`):* Resolved via the Tokio event loop (`--async`), multiplexing connections over `epoll` / `kqueue` on a fixed thread pool.
2. **Synchronous Socket I/O (in Sync Mode):**
   - Connection threads in the sync server use blocking `std::io::BufReader`. Running in `--async` mode eliminates per-connection thread pinning.
3. **Single-Node In-Memory Execution Queues & Result Store:**
   - While authentication keys (`auth:<key_hash>`) and penalty states (`penalty:<key_hash>`) are persisted in Redis (L2 `DB 1`), scheduler queues and `ResultStore` reside in process memory. If the broker crashes, uncollected job results and pending queues are lost.
4. **Container Startup Latency (Mitigated by Pre-Warmed Pool):**
   - *Status:* The engine features an asynchronous `PrewarmedContainerPool` ("Bake and Burn") paired with a pre-baked bytecode Docker image (`distjob-python:latest`), reducing container initialization latency from ~500ms down to sub-20ms.
   - *Remaining Boundary:* Under sustained bursts exceeding the pool replenishment rate, the engine falls back to on-demand `docker run` (~300ms–800ms). Further acceleration requires kernel-bypass hypervisors (e.g., AWS Firecracker microVMs).

---

6. Production Roadmap & Future Work



1. **Phase 1: Async Event-Driven Network Ingress (`tokio`):** *(Implemented in `src/server_async.rs`)* Provides an asynchronous event reactor alongside the synchronous server. Idle connections sleep directly in kernel `epoll` / `kqueue` tables on a fixed pool of threads matching physical CPU cores.
2. **Phase 2: Pre-Warmed Pool & MicroVM Acceleration:** *(Pre-Warmed Pool implemented in `src/sandbox.rs`)* Following the implementation of the Pre-Warmed Pool ("Bake and Burn"), the next horizon is transitioning from Docker/runc containers to AWS Firecracker microVMs over KVM for sub-5ms boot times and hardware-level virtualization.
3. **Phase 3: Two-Tier L1/L2 Authentication & Poll Rate-Limiting:** *(Implemented in `src/auth.rs`)* Server-issued `secret_key` tokens hashed via SHA-256 across a 32-shard in-process L1 heap cache and L2 Redis (`DB 1`), enforcing cross-tenant job ownership and demoting polling spammers to L2 penalty probation.
4. **Phase 4: Distributed Job Queuing & Persistence:** Decouple broker ingress nodes from execution workers using a distributed queue (e.g., Redis Streams, NATS, or a Raft log) and persist result records in durable distributed storage for high availability.
5. **Phase 5: Explicit Cancellation Protocol:** Implement a `CANCEL <secret_key> <job_id>` command allowing clients to abort runaway jobs voluntarily before their wall-clock timeout expires, freeing worker slots immediately.